

# ROTIR Python Port: Implementation Specification

**Purpose:** This document is the complete spec for building the ROTIR Python/JAX port. It's written for an AI coding assistant (Claude Sonnet in Claude Code) to implement module by module. Read this entire document before writing any code.

**Context:** ROTIR is a Julia package for stellar surface reconstruction from optical interferometry data. We're porting it to Python with JAX for GPU acceleration, with a future goal of integrating NIFTy8 for Bayesian inference. The NIFTy integration is NOT part of this spec — it will come later once the forward model is validated.

**Reference files:** The original Julia source files are provided alongside this spec. Reference them for algorithms and math, but do NOT translate them line-by-line. The Python code should be idiomatic, JAX-compatible, and clean.

---

## Table of Contents

- [1. Project Structure](#)
  - [2. Dependencies](#)
  - [3. Data Structures](#)
  - [4. Module 1: HEALPix Tessellation](#)
  - [5. Module 2: Geometry — Base, Projection, Rotation](#)
  - [6. Module 3: Geometry — Rapid Rotator](#)
  - [7. Module 4: Geometry — Roche Lobe](#)
  - [8. Module 5: Forward Model — Polygon FT](#)
  - [9. Module 6: Forward Model — Observables and Chi-squared](#)
  - [10. Module 7: OIFITS I/O](#)
  - [11. Module 8: Regularization](#)
  - [12. Module 9: Reconstruction](#)
  - [13. Coordinate Conventions and Gotchas](#)
  - [14. Testing Strategy](#)
  - [15. Build Order](#)
- 

## 1. Project Structure

```
rotir_jax/
├── __init__.py
├── datatypes.py           # All dataclasses (Section 3)
├── tessellation/
│   ├── __init__.py
│   └── healpix.py        # HEALPix tessellation + vertex computation
├── geometry/
│   ├── __init__.py
│   ├── base.py           # Rotation, projection, create_star, visibility
│   ├── rapid_rotator.py  # Rapid rotator radii + von Zeipel
│   ├── roche.py          # Roche potential, root-finding, gravity
│   └── orbits.py         # Kepler equation, binary orbits, separation
├── forward_model/
│   ├── __init__.py
│   ├── polyft.py         # Polygon Fourier transform (polyflux + polyft)
│   └── observables.py    # cvis → V2, T3, chi2
├── io/
│   ├── __init__.py
│   └── oifits_reader.py  # OIFITS loading → OIData dataclass
├── regularization/
│   ├── __init__.py
│   └── regularizers.py   # TV, L2, MEM, etc.
├── reconstruction/
│   ├── __init__.py
│   └── optimizer.py      # L-BFGS-B wrapper, parametric fitting
├── tests/
│   ├── test_healpix.py
│   ├── test_geometry.py
│   ├── test_polyft.py
│   ├── test_forward_model.py
│   └── test_roundtrip.py
```

---

## 2. Dependencies

```
# requirements.txt
jax[cuda12]          # or jax[cpu] for CPU-only
jaxlib
numpy
scipy
healpy               # For cross-validation of our vertex computation
astropy              # For OIFITS reading
matplotlib            # For plotting (optional, not core)
```

**JAX usage rules:** - Use `jax.numpy` (aliased as `jnp`) for all array math in hot paths - Use `@jax.jit` on functions that will be called repeatedly (forward model, `chi2`) - Use `jax.vmap` for batching over pixels or UV points where applicable - Use `float32` by default (matches Julia code's `Float32` convention) - Pure NumPy is fine for one-time setup (tessellation, data loading) - Do NOT jit functions that contain Python control flow (if/else on array values)

---

## 3. Data Structures

File: `rotir_jax/datatypes.py`

```
"""Core data structures for ROTIR.
```

```
Design principles:
```

- Geometry parameters use a class hierarchy (base + geometry-specific subclasses)
- Runtime data (tessellation, stellar geometry, OI data) are plain dataclasses
- JAX arrays for anything entering the forward model
- NumPy arrays for one-time setup data

```
"""
```

```
from dataclasses import dataclass, field
from typing import Optional, Literal
import numpy as np
import jax.numpy as jnp
```

```
# =====
# Geometry parameter hierarchy
# =====
```

```
@dataclass
```

```
class GeometricParams:
```

```
    """Base parameters shared by all geometry types.
```

```
    Units:
```

```
        inclination: degrees (0 = pole-on, 90 = edge-on)
        position_angle: degrees (measured East of North)
        rotation_period: days
```

```
    """
```

```
    inclination: float          # degrees
```

```
    position_angle: float       # degrees
```

```
    rotation_period: float      # days
```

```
    # Limb darkening
```

```
    ld_type: Literal[1, 2, 3] = 1 # 1=linear, 2=quadratic, 3=Hestroffer
```

```
    ld1: float = 0.0
```

```
    ld2: float = 0.0
```

```
@dataclass
```

```
class SphereParams(GeometricParams):
```

```
    """Simple sphere."""
```

```
    surface_type: int = field(default=0, init=False)
```

```
    radius: float = 1.0 # mas
```

```

@dataclass
class EllipsoidParams(GeometricParams):
    """Triaxial ellipsoid."""
    surface_type: int = field(default=1, init=False)
    radius_x: float = 1.0 # mas
    radius_y: float = 1.0 # mas
    radius_z: float = 1.0 # mas
    tpole: float = 10000.0 # K (for von Zeipel)
    beta: float = 0.25 # gravity darkening exponent

@dataclass
class RapidRotatorParams(GeometricParams):
    """Rapidly rotating star (Roche model for single star rotation)."""
    surface_type: int = field(default=2, init=False)
    rpole: float = 1.0 # mas (polar radius)
    frac_escapevel: float = 0.0 #  $\omega/\omega_{\text{crit}}$ , dimensionless [0, 1)
    tpole: float = 10000.0 # K (for von Zeipel)
    beta: float = 0.25 # gravity darkening exponent

@dataclass
class RocheParams(GeometricParams):
    """Roche lobe geometry (single visible star in a binary).

    The companion is treated as a point mass. The Roche geometry
    is parameterized by orbital elements + mass ratio.

    Units:
        rpole: mas (polar radius of the visible star)
        a: mas (semi-major axis of the binary orbit)
        P: days (orbital period)
        T0: JD (time of periastron passage)
        All angles: degrees unless noted
    """
    surface_type: int = field(default=3, init=False)
    # Stellar
    rpole: float = 1.0 # mas (polar radius)
    tpole: float = 10000.0 # K
    beta: float = 0.25 # gravity darkening exponent
    # Binary orbital
    a: float = 5.0 # mas (semi-major axis)
    P: float = 100.0 # days (orbital period)
    T0: float = 0.0 # JD (periastron time)
    e: float = 0.0 # eccentricity
    omega: float = 0.0 # argument of periapsis (degrees)
    Omega: float = 0.0 # longitude of ascending node (degrees, NOT USED for single star)
    q: float = 1.0 # mass ratio M2/M1
    dP: float = 0.0 # period derivative (days/day)
    # Roche-specific
    fillout_factor: float = -1.0 # if > 0, use fillout instead of rpole to define surface
    # Async rotation: ratio of stellar rotation to orbital rotation
    # async_ratio = P / rotation_period
    # (rotation_period is inherited from GeometricParams)

# =====
# Tessellation
# =====

@dataclass
class Tessellation:
    """HEALPix tessellation of the unit sphere.

    Arrays:

```

unit\_xyz: (npix, 5, 3) – xyz coords of 4 vertices + center, on unit sphere  
unit\_spherical: (npix, 5, 3) – (r, theta, phi) for each vertex + center

Index convention for the 5 points per pixel:

0-3: four quadrilateral vertices (N, W, S, E in HEALPix convention)  
4: pixel center

Coordinate convention:

theta: colatitude [0, pi], 0 = North Pole  
phi: longitude [0, 2\*pi], increasing East  
 $x = \sin(\theta) * \cos(\phi)$   
 $y = \sin(\theta) * \sin(\phi)$   
 $z = \cos(\theta)$

"""

tessellation\_type: int # 0=HEALPix, 1=LatLong  
npix: int  
nside: int # HEALPix nside (nside = 2^n)  
n: int # HEALPix order (n in nside=2^n)  
unit\_xyz: np.ndarray # (npix, 5, 3) float32  
unit\_spherical: np.ndarray # (npix, 5, 3) float32

# =====  
# Stellar geometry (per-epoch, after rotation + projection)  
# =====

@dataclass

class StellarGeometry:

"""Geometry of a star at a specific epoch, after rotation and projection.

This is the result of: tessellation + params + epoch → rotated, projected star.

Arrays:

vertices\_xyz: (npix, 5, 3) – rotated xyz coordinates  
vertices\_spherical: (npix, 5, 3) – (r, theta, phi) before rotation  
normals: (npix, 3) – outward normal of each pixel face  
projx: (nvis, 4) – x-coords of visible pixel vertices on sky plane (mas)  
projy: (nvis, 4) – y-coords of visible pixel vertices on sky plane (mas)  
visible\_idx: (nvis,) – indices of visible pixels into the full npix array  
ldmap: (npix,) – limb darkening weight per pixel (full array)  
polyflux: (nvis,) – pixel projected areas (set later by polyft module)  
polyft: (nuv, nvis) complex – polygon FT matrix (set later)

"""

surface\_type: int  
npix: int  
nvis: int  
vertices\_xyz: jnp.ndarray # (npix, 5, 3)  
vertices\_spherical: np.ndarray # (npix, 5, 3)  
normals: jnp.ndarray # (npix, 3)  
visible\_idx: jnp.ndarray # (nvis,) int  
projx: jnp.ndarray # (nvis, 4)  
projy: jnp.ndarray # (nvis, 4)  
ldmap: jnp.ndarray # (npix,)  
epoch: float # time of this geometry  
# These are set later by setup\_polygon\_ft:  
polyflux: Optional[jnp.ndarray] = None # (nvis,)  
polyft: Optional[jnp.ndarray] = None # (nuv, nvis) complex64

# =====  
# Interferometric data  
# =====

@dataclass

class OIData:

"""Interferometric observables from one OIFITS file / epoch.

UV coordinates:

uv: (2, nuv) – (u, v) spatial frequencies in units that match the conversion factor:  $kx = uv[0,:] * (-\pi / (180*3600*1000))$  i.e., uv is in cycles/radian if you want, but typically the OIFITS stores them in meters and the wavelength scaling converts them. The key is: uv values here are ALREADY in the correct units such that the Julia convention holds:  $kx = uv[0,:] * (-\pi / (180*3600000))$  This means uv is in "1/mas" effective units.

ACTUALLY: OIFITS stores u,v in meters. The conversion to spatial frequency happens in setup\_polyft. The Julia code does:  
 $kx = data.uv[1,:] * \text{Float32}(-\pi / (180*3600000))$   
 $ky = data.uv[2,:] * \text{Float32}(-\pi / (180*3600000))$  # NOTE: see gotchas  
So uv should be stored as-is from OIFITS (in meters/wavelength = cycles/radian).

CORRECTION: Looking at the Julia code more carefully:  
uv is stored as u/lambda, v/lambda in units of 1/radian.  
The OIT00LS package pre-divides by wavelength.  
The conversion factor  $-\pi/(180*3600*1000)$  converts from 1/radian to 1/mas (with the pi factor for the sinc convention).

For our Python code: store uv in OIFITS native units (1/radian, already divided by wavelength), and apply the same conversion in setup\_polyft.

Indexing arrays (all 0-based in Python, were 1-based in Julia):

```
indx_v2: (nv2,) – index into cvs for each V2 measurement
indx_t3_1/2/3: (nt3,) – three baseline indices for each closure phase
"""
# V2 observables
v2: jnp.ndarray          # (nv2,)
v2_err: jnp.ndarray      # (nv2,)
nv2: int
# Closure phase observables
t3phi: jnp.ndarray       # (nt3,)
t3phi_err: jnp.ndarray   # (nt3,)
nt3: int
# T3 amplitude (optional, often not used in chi2)
t3amp: jnp.ndarray       # (nt3,)
t3amp_err: jnp.ndarray   # (nt3,)
# UV coordinates
uv: jnp.ndarray          # (2, nuv) – spatial frequencies
nuv: int
# Indexing: which UV points correspond to which observables
indx_v2: jnp.ndarray     # (nv2,) int – index into cvs array
indx_t3_1: jnp.ndarray   # (nt3,) int
indx_t3_2: jnp.ndarray   # (nt3,) int
indx_t3_3: jnp.ndarray   # (nt3,) int
# Metadata
mean_mjd: float = 0.0
filename: str = ""
```

### IMPORTANT notes on the data structures:

1. The StellarGeometry struct is mutable — polyflux and polyft are set after creation, because they depend on the UV coverage of the data. This mirrors the Julia pattern where setup\_oif! mutates the star struct.
  2. visible\_idx replaces the Julia index\_quads\_visible. It maps from the compact visible-pixel arrays (projx, projy, polyflux, polyft columns) back to the full npix array. This is essential: the brightness map x has npix elements, but only x[visible\_idx] enters the forward model.
  3. All index arrays (indx\_v2, etc.) are **0-based** in Python. The Julia code uses 1-based indexing. When porting the OIFITS reader, subtract 1 from all indices.
-

## 4. Module 1: HEALPix Tessellation

File: rotir\_jax/tessellation/healpix.py

### What it does

Creates a tessellation of the unit sphere where each pixel is a quadrilateral with 4 vertices. This is the foundation for everything — the vertices define the polygon shapes used in the Fourier transform.

### Key function

```
def tessellation_healpix(n: int) -> Tessellation:
    """Create HEALPix tessellation of order n.

    Args:
        n: HEALPix order. nside = 2^n, npix = 12 * nside^2.
           n=3 → 768 pixels, n=4 → 3072 pixels.

    Returns:
        Tessellation with unit_xyz and unit_spherical arrays.
    """
```

### Algorithm

**Step 1:** Use healpy for pixel centers. ``python import healpy as hp

nside = 2\*\*n npix = hp.nside2npix(nside)

## Get center (theta, phi) for each pixel in NESTED ordering

## CRITICAL: ROTIR uses NESTED ordering throughout, not RING

```
theta_center, phi_center = hp.pix2ang(nside, np.arange(npix), nest=True) ``
```

**Step 2:** Compute pixel vertices. This is the hard part — healpy does NOT provide vertex positions in nested ordering with the exact layout ROTIR needs. You MUST port the vertex computation from `pix2vec_nest` in `tessellation_healpix.jl`.

The Julia `pix2vec_nest` function (lines 380-490 of `tessellation_healpix.jl`) computes: - `vec_out`: (npix, 3) — center xyz - vertex: (npix, 4, 3) — four vertex xyz positions

The vertex ordering is: North, West, South, East (indices 0,1,2,3).

The algorithm works by: 1. Converting nested pixel index → face number + (ix, iy) within face 2. Computing ring number  $j_r$  and phi number  $j_p$  3. Determining if pixel is in equatorial, north polar, or south polar region 4. Computing  $z$  ( $\cos(\theta)$ ) and phi for center and vertices 5. Special handling for transition pixels at nside boundary

**IMPORTANT:** You can validate your vertex computation against `healpy.boundaries()`: ``python

## healpy provides vertices per pixel (in RING ordering), but you can convert

## This is for VALIDATION only — your actual code should compute them directly

```
vertices_hp = np.array([hp.boundaries(nside, i, step=1, nest=True).T for i in range(npix)])
```

## vertices\_hp shape: (npix, 4, 3)

## But NOTE: healpy vertex ordering may differ from Julia's N,W,S,E convention

**\*\*Step 3\*\*:** Convert to unit sphere coordinates.

## After computing vertices, ensure they're on the unit sphere

```
unit_xyz = np.zeros((npix, 5, 3), dtype=np.float32) unit_xyz[:, :4, :] = vertex_xyz # 4 vertices unit_xyz[:, 4, :] = center_xyz # center
```

## Compute spherical coordinates

**$r = 1$  for unit sphere**

**$\theta = \arccos(z) = \text{colatitude}$**

**$\phi = \arctan2(y, x)$ , wrapped to  $[0, 2\pi]$**

```
unit_spherical = np.zeros((npix, 5, 3), dtype=np.float32) unit_spherical[:, :, 0] = 1.0 # r = 1 unit_spherical[:, :, 1] = np.arctan2(
np.sqrt(unit_xyz[:, :, 0]**2 + unit_xyz[:, :, 1]**2), unit_xyz[:, :, 2]) # theta phi = np.arctan2(unit_xyz[:, :, 1], unit_xyz[:, :, 0]) unit_spherical[:, :, 2] = phi + 2*np.pi * (phi < 0) # phi in [0, 2pi)``
```

## Neighbor computation

```
def get_neighbors(n: int) -> list:
    """Get neighbor indices for each pixel.

    Uses healpy.get_all_neighbours in nested ordering.

    Returns:
        List of arrays, neighbors[i] = array of neighbor indices for pixel i.
        Length is 8 for most pixels, 7 for some corner pixels.
        Indices where neighbor = -1 should be excluded.
    """
    nside = 2**n
    npix = hp.nside2npix(nside)
    neighbors = []
    for i in range(npix):
        nb = hp.get_all_neighbours(nside, i, nest=True)
        neighbors.append(nb[nb >= 0]) # remove -1 entries
    return neighbors
```

## TV regularization matrices

```
def tv_regularization_matrices(n: int, visible_idx: Optional[np.ndarray] = None):
    """Build sparse gradient and Laplacian matrices for TV regularization.

    Args:
        n: HEALPix order
        visible_idx: if provided, restrict to visible pixels only

    Returns:
        grad_matrix: sparse (npix, npix) gradient operator
        laplacian: sparse (npix, npix) = grad_matrix.T @ grad_matrix
        neighbors: list of neighbor arrays
    """
```

Port the logic from `tv_neighbours_healpix` (lines 931-952 of `tessellation_healpix.jl`). The gradient matrix  $\nabla$  has:  $-\nabla[k, k] = \text{number\_of\_neighbors}(k) - \nabla[\text{neighbor}, k] = -1$  for each neighbor

---

## 5. Module 2: Geometry — Base

File: rotir\_jax/geometry/base.py

## Rotation matrix

```
def rotation_matrix(phase_angle: float, inclination_rad: float, pa_rad: float) -> jnp.ndarray:
    """Compound rotation matrix: rotation phase → inclination → position angle.
```

This matches the Julia `rot\_vertex` function.

Args:

```
    phase_angle: rotation phase in radians (2*pi*t/P for single stars,
                or 2*pi*(t-T0)/P for Roche)
    inclination_rad: inclination in radians
    pa_rad: position angle in radians
```

Returns:

```
    (3, 3) rotation matrix. Apply as: xyz_rotated = xyz @ R
    (note: row-vector convention, matching Julia's reshape-multiply pattern)
    """
```

The Julia code (geometry.jl, line 120-131):

```
julia function rot_vertex(angle_r1, angle_r2, angle_r3) c1 =
cos(angle_r1); s1 = sin(angle_r1) c2 = cos(angle_r2); s2 = sin(angle_r2) c3 = cos(angle_r3); s3 =
sin(angle_r3) dcm = [-s1*c2*s3+c1*c3 s1*c3*c2+c1*s3 -s1*s2; -c1*c2*s3-s1*c3 c1*c3*c2-s1*s3 -c1*s2
; -s2*s3 s2*c3 c2 ]; return dcm end
```

And it's applied as (geometry.jl, line 166):

```
julia reshape(reshape(xyz, (npix*5, 3))*compound_rotation, (npix,
5, 3))
```

So: `rotated = flat_xyz @ dcm`, where `flat_xyz` is `(npix*5, 3)` and `dcm` is `(3, 3)`.

**This means the rotation matrix acts on ROW vectors** (right-multiply). Implement accordingly.

## Create star

```
def create_star(
    tessellation: Tessellation,
    params: GeometricParams,
    epoch: float,
    radii: Optional[jnp.ndarray] = None,
) -> StellarGeometry:
    """Create a StellarGeometry for one epoch.
```

Pipeline:

1. Compute radii  $r(\theta, \phi)$  based on geometry type
2. Scale `unit_xyz` by radii: `xyz = r * unit_xyz`
3. Rotate: `xyz_rot = flat(xyz) @ rotation_matrix`
4. Compute normals from cross products of diagonals
5. Determine visibility (`normal_z > 0`)
6. Extract projected  $(x, y)$  for visible pixels
7. Compute limb darkening map

Args:

```
    tessellation: Base tessellation (unit sphere)
    params: Geometry parameters (SphereParams, RapidRotatorParams, etc.)
    epoch: Time of this observation (days, or JD depending on context)
    radii: Optional pre-computed radii array (npix, 5). If None, compute
           from params. This allows Roche radii to be computed externally.
```

Returns:

```
    StellarGeometry with all fields populated except polyflux/polyft.
    """
```

**Step-by-step implementation** (reference: `create_star` in `geometry.jl`, lines 181-211):

### 1. Compute radii:

- Sphere: `r = params.radius * np.ones((npix, 5))`
- Ellipsoid: scale `unit_xyz` by  $(r_x, r_y, r_z)$ , then compute `r = norm`



- Rapid rotator: call `update_radii_rapidrot(tessellation, params)`
- Roche: `r` is passed in via `radii` argument (computed externally)

2. **Scale to 3D coordinates:** `xyz = r[:, :, None] * tessellation.unit_xyz`

3. **Rotate:** `python if params.surface_type == 3: # Roche phase = 2 * np.pi * (epoch - params.T0) / params.P else: phase = 2 * np.pi * epoch / params.rotation_period`

`R = rotation_matrix(phase, params.inclination * np.pi/180, params.position_angle * np.pi/180) xyz_flat = xyz.reshape(-1, 3) xyz_rot = (xyz_flat @ R).reshape(npix, 5, 3)```

4. **Compute normals** (cross product of diagonals AC and BD): `python vecAC = xyz_rot[:, 2, :] - xyz_rot[:, 0, :] # vertex 2 - vertex 0 vecBD = xyz_rot[:, 3, :] - xyz_rot[:, 1, :] # vertex 3 - vertex 1 normals = jnp.cross(vecAC, vecBD) normals = normals / jnp.linalg.norm(normals, axis=1, keepdims=True)`

5. **Visibility:** `visible_mask = normals[:, 2] > 0` (z-component > 0 means facing observer)

6. **Projection:** `python visible_idx = jnp.where(visible_mask)[0] projx = xyz_rot[visible_idx, :4, 0] # (nvis, 4) - x-coords on sky projy = xyz_rot[visible_idx, :4, 1] # (nvis, 4) - y-coords on sky`

7. **Limb darkening:** `mu = jnp.maximum(normals[:, 2], 0.0) python if params.ld_type == 1: ldmap = 1.0 - params.ld1 * (1.0 - mu) elif params.ld_type == 2: ldmap = 1.0 - params.ld1 * (1.0 - mu) - params.ld2 * (1.0 - mu)**2 elif params.ld_type == 3: ldmap = mu ** params.ld1`

## Multi-epoch creation

```
def create_star_multiePOCHs(
    tessellation: Tessellation,
    params: GeometricParams,
    epochs: list[float],
) -> list[StellarGeometry]:
    """Create StellarGeometry for each epoch.
```

For Roche geometry with eccentric orbits, the radii change per epoch because the binary separation  $D$  changes. For other geometries, only the rotation angle changes.

```
"""
stars = []
for epoch in epochs:
    if params.surface_type == 3: # Roche
        D = compute_separation(params, epoch)
        radii = update_roche_radii(tessellation, params, D)
        star = create_star(tessellation, params, epoch, radii=radii)
    else:
        star = create_star(tessellation, params, epoch)
    stars.append(star)
return stars
```

## 6. Module 3: Rapid Rotator

File: `rotir_jax/geometry/rapid_rotator.py`

### Radii computation

```
def f_rapid_rot(x: jnp.ndarray) -> jnp.ndarray:
    """Rapid rotation radius function.

     $r(\theta) = r_{\text{pole}} * f(\omega * \sin(\theta))$ 
    where  $f(x) = 3 * \cos((\pi + \arccos(x))/3) / x$ 

    Handle  $x \rightarrow 0$  limit:  $f(0) = 1$  (sphere at poles).
    """
    # Use jnp.where to handle the  $x=0$  singularity
    safe_x = jnp.where(jnp.abs(x) < 1e-10, 1e-10, x)
    result = 3.0 * jnp.cos((jnp.pi + jnp.arccos(jnp.clip(safe_x, -1, 1))) / 3.0) / safe_x
    return jnp.where(jnp.abs(x) < 1e-10, 1.0, result)
```

```
def update_radII_rapidrot(tessellation: Tessellation, params: RapidRotatorParams) -> np.ndarray:
    """Compute radii for a rapidly rotating star.

    Returns:
        r: (npix, 5) array of radii in mas.
    """
    omega = params.frac_escapevel
    theta = tessellation.unit_spherical[:, :, 1] # (npix, 5) colatitudes
    r = params.rpole * f_rapid_rot(omega * np.sin(theta))
    return r.astype(np.float32)
```

## Von Zeipel temperature map

```
def temperature_map_vonZeipel_rapid_rotator(
    params: RapidRotatorParams,
    star: StellarGeometry,
    GM: float = 1.0,
) -> jnp.ndarray:
    """Compute effective temperature map from gravity darkening.

    T_eff(theta) = T_pole * (g(theta) / g_pole)^beta

    Returns:
        tmap: (npix,) temperature array.
    """
```

Reference: `temperature_map_vonZeipel_rapid_rotator` in `geometry_rapidrotator.jl`, lines 80-98.

The gravity computation uses:  $r_{\theta}$  = distance from center for each pixel (from rotated xyz, center vertex index 4) -  $\theta$  = colatitude from spherical coords (pre-rotation) -  $\omega_{\text{crit}} = \sqrt{8GM / (27r_{\text{pole}}^3)}$  -  $\omega = \text{frac\_escapevel} * \omega_{\text{crit}}$  -  $g_r = -GM/r^2 + r*(\omega*\sin(\theta))^2$  -  $g_{\theta} = \omega^2 * r * \sin(\theta) * \cos(\theta)$  -  $g = \sqrt{g_r^2 + g_{\theta}^2}$  -  $g_{\text{pole}} = GM / r_{\text{pole}}^2$

## 7. Module 4: Roche Lobe

File: `rotir_jax/geometry/roche.py`

This is the most complex geometry module. It computes the Roche equipotential surface for a star in a binary system.

### Roche potential

```
def compute_potential_primary(
    r: float, D: float, theta: float, phi: float, q: float, async_ratio: float
) -> tuple[float, float, float]:
    """Compute Roche potential and its first two derivatives for the primary.

    The primary is at origin (0,0,0), secondary at (D,0,0) in the co-rotating frame.
    All lengths are dimensionless (divided by semi-major axis a).

    Args:
        r: dimensionless radius (r/a)
        D: dimensionless instantaneous separation ((1-e^2)/(1+e*cos(v)))
        theta: colatitude [0, pi]
        phi: longitude [0, 2*pi]
        q: mass ratio M2/M1
        async_ratio: stellar rotation / orbital rotation = P_orb / P_rot

    Returns:
        Omega: potential value
        dOmega: dΩ/dr (first derivative)
        ddOmega: d²Ω/dr² (second derivative)
```

Reference: `geometry_rochelobe.jl` lines 130-137.

```

"""
lam = sin(theta) * cos(phi) # λ = direction cosine toward secondary
nu = cos(theta)             # ν = direction cosine toward pole

# Distance from secondary
dist_sq = D**2 + r**2 - 2*r*lam*D
dist = sqrt(dist_sq)

Omega = 1/r + q/dist - q*r*lam*D + async_ratio**2 * (1+q) * r**2 * (1-nu**2) / 2

dOmega = (-1/r**2
          - q*(r - lam*D) / dist**3
          - q*lam*D
          + async_ratio**2 * (1+q) * r * (1-nu**2))

ddOmega = (2/r**3
          + 3*(D*lam - r)**2 * q / dist**5
          - q / dist**3
          + async_ratio**2 * (1+q) * (1-nu**2))

return Omega, dOmega, ddOmega

```

Similarly implement `compute_potential_secondary` (lines 140-148 of `geometry_rochelobe.jl`). Note the different formula — the secondary is at  $(D,0,0)$  so the signs and terms change.

## Root finding: Halley's method

```

def halley_solve(
    r0: float, target_potential: float, D: float,
    theta: float, phi: float, q: float, async_ratio: float,
    potential_fn, # compute_potential_primary or _secondary
    max_iter: int = 20, tol: float = 1e-5,
) -> float:
    """Find r such that potential(r) = target_potential using Halley's method.

    Halley's method uses f, f', f'' for cubic convergence:
        step = 2*f*f' / (2*f'^2 - f*f'')
        r_new = r - step

    Reference: geometry_rochelobe.jl lines 198-216.
    """
    r = r0
    for _ in range(max_iter):
        f, g, h = potential_fn(r, D, theta, phi, q, async_ratio)
        halley_step = 2 * (f - target_potential) * g / (2 * g**2 - (f - target_potential) * h)
        r -= halley_step
        if abs(halley_step) < tol:
            break
    return r

```

**IMPORTANT:** This function is NOT jit-able because of the while loop with early termination. That's fine — it's called during geometry setup, not in the inner forward model loop. If you need to vectorize it over pixels, use Python loops or `np.vectorize` (not JAX).

## Surface potential from parameters

```

def get_surface_potential(
    rpole_a: float, D: float, q: float, async_ratio: float,
    fillout_factor: float = -1.0,
    secondary: bool = False,
) -> tuple[float, float]:
    """Compute the target surface potential and initial radius guess.

    Two modes:
    1. fillout_factor > 0: Compute potential from L1 point and fillout
    2. fillout_factor <= 0: Compute potential from rpole (pole defines the surface)

```

Returns:

```

    pot_surface: target potential value
    r_init: initial guess for radius (for root-finder)
    """

```

Reference: `get_surface_potential` in `geometry_rochelobe.jl` lines 48-64.

Key formulas: - L1 point: find where  $d\Omega/dr = 0$  along the line connecting the stars ( $\theta=\pi/2$ ,  $\phi=0$  for primary) - Fillout: `pot_surface = (pot_L1 +  $q^2/2(1+q)$ ) / fillout -  $q^2/2(1+q)$`  (Leahy eq. 6) - From rpole: evaluate potential at ( $r=r_{\text{pole}}/a$ ,  $\theta=0$ ,  $\phi=0$ )

## Update radii for all pixels

```

def update_roche_radii(
    tessellation: Tessellation, params: RocheParams, D: float,
) -> np.ndarray:
    """Compute Roche surface radii for all tessellation pixels.

    For each pixel (theta, phi), solve: Potential(r, theta, phi) = pot_surface

    Args:
        tessellation: HEALPix tessellation
        params: Roche parameters
        D: dimensionless instantaneous separation

    Returns:
        r: (npix, 5) radii in mas (= dimensionless_r * a)
    """
    async_ratio = params.P / params.rotation_period
    q = params.q
    a = params.a
    rpole_a = params.rpole / a

    use_fillout = params.fillout_factor > 0
    pot_surface, r_init = get_surface_potential(
        rpole_a, D, q, async_ratio,
        fillout_factor=params.fillout_factor if use_fillout else -1.0,
    )

    npix = tessellation.npix
    r = np.zeros((npix, 5), dtype=np.float32)
    for ii in range(npix):
        for jj in range(5):
            theta = tessellation.unit_spherical[ii, jj, 1]
            phi = tessellation.unit_spherical[ii, jj, 2]
            r_dimless = halley_solve(
                r_init, pot_surface, D, theta, phi, q, async_ratio,
                compute_potential_primary,
            )
            r[ii, jj] = a * r_dimless

    return r

```

**Performance note:** This loop over  $\text{npix} \times 5$  is the bottleneck for Roche geometry. For  $n=3$  (768 pixels  $\times$  5 = 3840 solves) it should be ~1 second in Python. For  $n=4$  (3072  $\times$  5 = 15360 solves) it could be 3-4 seconds. This is acceptable for now. Future optimization: vectorize with JAX using fixed-iteration Newton (no early termination), or use multiprocessing.

## Gravity computation (for von Zeipel)

```

def compute_gravity_primary(
    r, theta, phi, D, q, async_ratio
) -> jnp.ndarray:
    """Compute surface gravity magnitude for primary star.

```

Reference: `geometry_rochelobe.jl` lines 219-233.  
All inputs can be arrays (vectorized).

Returns:

```

    |g|: gravity magnitude (dimensionless)
    """
    lam = jnp.cos(phi) * jnp.sin(theta)
    mu = jnp.sin(phi) * jnp.sin(theta)
    nu = jnp.cos(theta)
    x = r * lam
    y = r * mu
    z = r * nu

    rho = jnp.sqrt(((D - x)**2 + y**2 + z**2)**(-3))
    mu_inv = r**(-3)

    gx = -x * mu_inv + q * (D - x) * rho + async_ratio**2 * (1+q) * x - q * D
    gy = -y * mu_inv - q * y * rho + async_ratio**2 * (1+q) * y
    gz = -z * mu_inv - q * z * rho

    return jnp.sqrt(gx**2 + gy**2 + gz**2)

```

---

## 8. Module 5: Orbits

File: rotir\_jax/geometry/orbits.py

### Key functions

```

def compute_eccentric_anomaly(params: RocheParams, epoch: float) -> float:
    """Solve Kepler's equation  $M = E - e \sin(E)$  via Newton-Raphson.

    Reference: orbits.jl lines 1-27, 130-147.
    """

def compute_true_anomaly(params: RocheParams, epoch: float) -> float:
    """Compute true anomaly from eccentric anomaly.

    Uses Broucke-Cefola formula to avoid singularity near  $\pm\pi$ :
     $\beta = e / (1 + \sqrt{1-e^2})$ 
     $v = E + 2 \arctan(\beta \sin(E) / (1 - \beta \cos(E)))$ 

    Reference: orbits.jl lines 151-162.
    """

def compute_separation(params: RocheParams, epoch: float) -> float:
    """Compute dimensionless instantaneous binary separation D.

     $D = 1 - e \cos(E)$ , where E is eccentric anomaly.
    Multiply by `a` to get physical separation.

    Reference: orbits.jl lines 120-127.
    """

```

These are straightforward numerical functions. For circular orbits ( $e=0$ ),  $D=1$  always.

---

## 9. Module 5: Polygon Fourier Transform

File: rotir\_jax/forward\_model/polyft.py

**This is the mathematical core of the forward model.** Get this right and everything else follows.

### Polyflux (projected pixel areas)

```

@jax.jit
def compute_polyflux(projx: jnp.ndarray, projy: jnp.ndarray) -> jnp.ndarray:
    """Compute projected area of each visible pixel (shoelace formula).

```

For a quadrilateral with vertices  $(x_1, y_1)$ ,  $(x_2, y_2)$ ,  $(x_3, y_3)$ ,  $(x_4, y_4)$ :

$$\text{area} = 0.5 * |x_1 y_2 - x_2 y_1 + x_2 y_3 - x_3 y_2 + x_3 y_4 - x_4 y_3 + x_4 y_1 - x_1 y_4|$$

Args:

projx: (nvis, 4) x-coordinates of pixel vertices on sky plane (mas)  
 projy: (nvis, 4) y-coordinates of pixel vertices on sky plane (mas)

Returns:

polyflux: (nvis,) projected area per pixel

Reference: oichi2\_spheroid.jl lines 52-67 (setup\_polyflux\_single).

"""

# Shoelace formula for quadrilateral

```
polyflux = 0.5 * (
    projx[:, 0] * projy[:, 1] - projx[:, 1] * projy[:, 0] +
    projx[:, 1] * projy[:, 2] - projx[:, 2] * projy[:, 1] +
    projx[:, 2] * projy[:, 3] - projx[:, 3] * projy[:, 2] +
    projx[:, 3] * projy[:, 0] - projx[:, 0] * projy[:, 3]
)
return polyflux
```

## Polygon Fourier Transform matrix

@jax.jit

```
def compute_polyft(
    uv: jnp.ndarray,
    projx: jnp.ndarray,
    projy: jnp.ndarray,
) -> jnp.ndarray:
    """Compute the polygon Fourier transform matrix.
```

For each UV point and each pixel, computes the exact Fourier transform of the quadrilateral pixel, using the analytical formula for the FT of a polygon (sum over edges).

Args:

uv: (2, nuv) spatial frequencies. These are  $u/\lambda$ ,  $v/\lambda$  in 1/radian units.  
 projx: (nvis, 4) x-coords of pixel vertices (mas)  
 projy: (nvis, 4) y-coords of pixel vertices (mas)

Returns:

polyft: (nuv, nvis) complex64 – the FT matrix.  
 cvis = polyft @ brightness / flux

CRITICAL SIGN CONVENTION:

```
kx = uv[0, :] * (-pi / (180 * 3600 * 1000))
ky = uv[1, :] * (-pi / (180 * 3600 * 1000))
```

In the UNCOMMENTED production code (oichi2\_spheroid.jl line 118), BOTH kx and ky use the SAME sign: negative.

WARNING: Some COMMENTED-OUT versions in the Julia code use +pi for ky. The production version uses -pi for BOTH. Follow the production version.

Math for one edge (i→j) of the polygon:

```
dx = x_j - x_i
dy = y_j - y_i
cx = x_j + x_i    (center, doubled)
cy = y_j + y_i
```

```
k_dot_d = kx * dx + ky * dy
k_dot_c = kx * cx + ky * cy
cross    = ky * dx - kx * dy
```

```
edge_contribution = sinc(k_dot_d) * exp(-i*pi*k_dot_c) * cross
```

Sum over 4 edges, then multiply by:  $-i/(2\pi) / (k_x^2 + k_y^2)$

NOTE on sinc: Julia's  $\text{sinc}(x) = \sin(\pi x)/(\pi x)$ . JAX's  $\text{jnp.sinc}(x)$  is the same. The argument to sinc is  $k\_dot\_d$  (NOT  $k\_dot\_d/\pi$ ).

Reference: `oichi2_spheroid.jl` lines 118-148 (`setup_polyft_single`).

```
"""
# Convert UV to spatial frequencies in 1/mas
scale = jnp.float32(-jnp.pi / (180.0 * 3600.0 * 1000.0))
kx = uv[0, :] * scale # (nuv,)
ky = uv[1, :] * scale # (nuv,)

k2 = kx * kx + ky * ky # (nuv,)

nuv = uv.shape[1]
nvis = projx.shape[0]

# Initialize output
polyft = jnp.zeros((nuv, nvis), dtype=jnp.complex64)

# Loop over 4 edges: (0→1), (1→2), (2→3), (3→0)
for i in range(4):
    j = (i + 1) % 4

    dx = projx[:, j] - projx[:, i] # (nvis,)
    dy = projy[:, j] - projy[:, i]
    cx = projx[:, j] + projx[:, i]
    cy = projy[:, j] + projy[:, i]

    # Broadcasting: kx is (nuv,), dx is (nvis,)
    # Result shapes: (nuv, nvis)
    k_dot_d = kx[:, None] * dx[None, :] + ky[:, None] * dy[None, :]
    k_dot_c = kx[:, None] * cx[None, :] + ky[:, None] * cy[None, :]
    cross = ky[:, None] * dx[None, :] - kx[:, None] * dy[None, :]

    polyft += jnp.sinc(k_dot_d) * jnp.exp(-1j * jnp.pi * k_dot_c) * cross

# Apply prefactor
prefactor = -1j / (2.0 * jnp.pi)
polyft = prefactor * polyft / k2[:, None]

return polyft
```

**CRITICAL JAX NOTE:** The nested loop over 4 edges is fine because it's a static loop (unrolled by the JIT compiler). The broadcasting approach `kx[:, None] * dx[None, :]` creates `(nuv, nvis)` intermediate arrays. For `nuv=10000`, `nvis=3000`, each intermediate is ~120 MB in float32. This should be fine for GPU but watch memory on CPU. If memory is an issue, chunk over UV points.

**GOTCHA:** The  $k_2$  denominator is zero when  $k_x=k_y=0$ . This corresponds to the zero-frequency (total flux) baseline, which typically doesn't appear in real interferometric data. But if it does, handle it: `python k2 = jnp.where(k2 == 0, 1.0, k2) # avoid division by zero`

## The $k_2=0$ row of polyft will be garbage, but it won't be used

...

### Setup function

```
def setup_polygon_ft(star: StellarGeometry, data: OIData) -> StellarGeometry:
    """Compute polyflux and polyft for a star given data, and attach to star.
```

This is the equivalent of Julia's `setup_oi`!

Args:

star: StellarGeometry (will be mutated to add polyflux, polyft)  
data: OIData with UV coordinates

Returns:

```

    star: same object with polyflux and polyft set
"""
star.polyflux = compute_polyflux(star.projx, star.projy)
star.polyft = compute_polyft(data.uv, star.projx, star.projy)
return star

```

---

## 10. Module 6: Observables and Chi-squared

File: rotir\_jax/forward\_model/observables.py

### Complex visibilities from brightness

```

@jax.jit
def brightness_to_cvis(
    brightness: jnp.ndarray,
    visible_idx: jnp.ndarray,
    polyflux: jnp.ndarray,
    polyft: jnp.ndarray,
) -> jnp.ndarray:
    """Compute complex visibilities from surface brightness.

    Args:
        brightness: (npix,) surface brightness for ALL pixels
        visible_idx: (nvis,) indices of visible pixels
        polyflux: (nvis,) pixel projected areas
        polyft: (nuv, nvis) polygon FT matrix

    Returns:
        cvis: (nuv,) complex visibilities (normalized by total flux)
    """
    x_vis = brightness[visible_idx] # (nvis,)
    flux = jnp.dot(polyflux, x_vis) # scalar: total flux
    cvis = polyft @ x_vis / flux    # (nuv,)
    return cvis

```

### Observables

```

def mod360(x: jnp.ndarray) -> jnp.ndarray:
    """Wrap angle difference to [-180, 180] degrees."""
    return ((x + 180) % 360 + 360) % 360 - 180

@jax.jit
def cvis_to_observables(
    cvis: jnp.ndarray,
    indx_v2: jnp.ndarray,
    indx_t3_1: jnp.ndarray,
    indx_t3_2: jnp.ndarray,
    indx_t3_3: jnp.ndarray,
) -> tuple[jnp.ndarray, jnp.ndarray, jnp.ndarray]:
    """Compute V2, T3amp, T3phi from complex visibilities.

    Args:
        cvis: (nuv,) complex visibility
        indx_*: index arrays mapping observables to UV points

    Returns:
        v2_model: (nv2,) squared visibility amplitudes
        t3amp_model: (nt3,) triple amplitudes
        t3phi_model: (nt3,) closure phases in degrees
    """
    # V2
    v2_model = jnp.abs(cvis[indx_v2])**2

```



```
# Bispectrum (triple product)
t3 = cvis[indx_t3_1] * cvis[indx_t3_2] * cvis[indx_t3_3]
t3amp_model = jnp.abs(t3)
t3phi_model = jnp.angle(t3) * (180.0 / jnp.pi) # degrees

return v2_model, t3amp_model, t3phi_model
```

## Chi-squared

```
@jax.jit
def compute_chi2(
    brightness: jnp.ndarray,
    star: StellarGeometry,
    data: OIData,
) -> tuple[float, float, float]:
    """Compute chi-squared for V2, T3amp, T3phi.

    Returns:
        chi2_v2, chi2_t3amp, chi2_t3phi
    """
    cvis = brightness_to_cvis(brightness, star.visible_idx, star.polyflux, star.polyft)
    v2_model, t3amp_model, t3phi_model = cvis_to_observables(
        cvis, data.indx_v2, data.indx_t3_1, data.indx_t3_2, data.indx_t3_3
    )

    chi2_v2 = jnp.sum(((v2_model - data.v2) / data.v2_err)**2)
    chi2_t3amp = jnp.sum(((t3amp_model - data.t3amp) / data.t3amp_err)**2)
    chi2_t3phi = jnp.sum(((mod360(t3phi_model - data.t3phi) / data.t3phi_err)**2)

    return chi2_v2, chi2_t3amp, chi2_t3phi
```

## Gradient of chi-squared

**Option A:** Use JAX autodiff (recommended for initial implementation): ``python def chi2\_total(brightness, star, data): c2\_v2, c2\_t3a, c2\_t3p = compute\_chi2(brightness, star, data) return c2\_v2 + c2\_t3a + c2\_t3p

chi2\_grad = jax.grad(chi2\_total) ``

**Option B:** Analytic gradient (for performance, later). The Julia code has explicit gradient computation in `spheroid_chi2_fg` (`oichi2_spheroid.jl` lines 213-233). This is significantly faster because it avoids backward-mode AD overhead. Port it later if autodiff is too slow.

**IMPORTANT:** The Julia analytic gradient includes a flux-normalization correction (line 228): `julia g[indx] = (gsum .- dot(x[indx],gsum)*star.polyflux / flux) / flux` This accounts for the fact that `cvis` is normalized by total flux, so the gradient of `cvis` w.r.t. `x` includes a flux correction term. If using JAX autodiff, this is handled automatically. If porting the analytic gradient, include this correction.

---

## 11. Module 7: OIFITS I/O

File: `rotir_jax/io/oifits_reader.py`

### Reading OIFITS files

Use `astropy.io.fits` to read OIFITS files. The OIFITS format stores data in FITS binary table extensions: - `OI_VIS2`: squared visibilities - `OI_T3`: triple products (closure phases) - `OI_VIS`: complex visibilities (not always present) - `OI_WAVELENGTH`: wavelength table - `OI_ARRAY`: array configuration

```
def read_oifits(filename: str) -> OIData:
    """Read an OIFITS file and return an OIData structure.
```

Key steps:

1. Read `OI_WAVELENGTH` to get wavelength(s)
2. Read `OI_VIS2` for squared visibilities
3. Read `OI_T3` for closure phases
4. Build UV coordinate array

## 5. Build index arrays mapping V2 and T3 to UV points

The index arrays are the trickiest part. Each V2 measurement uses one baseline (one UV point). Each T3 measurement uses three baselines. The indices map from observable → UV point in the combined UV array.

**CRITICAL:** Convert from OIFITS 1-based STA\_INDEX to 0-based Python indexing.

Returns:

OIData with all arrays as jnp arrays (float32 for data, int32 for indices).

```
"""
```

**NOTE:** The Julia OITools package does significant preprocessing to build the index arrays. You may want to use the `pyoifits` package as an alternative if available, or look at how `oitools` for Python (if it exists) handles this. The key is getting `indx_v2`, `indx_t3_1/2/3` correct — these map each observable to the correct row in the UV array.

**UV convention:** OIFITS stores UC00RD and VC00RD in meters. These get divided by wavelength to give spatial frequencies in cycles/radian. The Julia code's `data.uv` is already in spatial-frequency units (cycles/radian or similar). Make sure your reader performs this division.

---

## 12. Module 8: Regularization

File: `rotir_jax/regularization/regularizers.py`

### Total variation

```
def total_variation_l2(x: jnp.ndarray, grad_matrix) -> tuple[float, jnp.ndarray]:
    """L2 total variation:  $||\nabla x||^2$ 

    Args:
        x: (npix,) brightness map
        grad_matrix: sparse gradient matrix from healpix module

    Returns:
        tv_value: scalar
        tv_grad: (npix,) gradient
    """
    # Convert sparse matrix to JAX if needed
    gx = grad_matrix @ x
    tv_f = jnp.sum(gx**2)
    tv_g = 2 * (grad_matrix.T @ gx)
    return tv_f, tv_g
```

### Maximum entropy

```
def max_entropy(x: jnp.ndarray, eps: float = 1e-9) -> tuple[float, jnp.ndarray]:
    """Maximum entropy regularization:  $\sum(x/\text{mean}(x) * \log(x/\text{mean}(x)))$ 

    Reference: oichi2_spheroid.jl lines 385-397.
    """
    xm = x / (jnp.mean(x) + eps)
    f = jnp.sum(xm * jnp.log(xm + eps))
    # Gradient - see Julia code for the exact expression
    g = ((jnp.mean(x) - x / len(x)) / (jnp.mean(x)**2 + eps)) * (jnp.log(xm + eps) + 1)
    return f, g
```

---

## 13. Module 9: Reconstruction

File: `rotir_jax/reconstruction/optimizer.py`

### Image reconstruction with L-BFGS-B

```

def reconstruct(
    x_start: jnp.ndarray,
    stars: list[StellarGeometry],
    data: list[OIData],
    regularizers: list = None,
    lower: float = 0.0,
    upper: float = float('inf'),
    maxiter: int = 200,
) -> jnp.ndarray:
    """Reconstruct surface brightness using L-BFGS-B optimization.

    This is the equivalent of Julia's image_reconstruct_oi.

    Args:
        x_start: (npix,) initial brightness guess
        stars: list of StellarGeometry (one per epoch)
        data: list of OIData (one per epoch)
        regularizers: list of (name, weight, extra_args) tuples
        lower: lower bound on brightness
        upper: upper bound on brightness
        maxiter: maximum iterations

    Returns:
        x_sol: (npix,) optimized brightness map
    """
    from scipy.optimize import minimize

    def objective(x_np):
        x = jnp.array(x_np, dtype=jnp.float32)
        # Sum chi2 over epochs
        total_chi2 = 0.0
        for star, datum in zip(stars, data):
            c2_v2, c2_t3a, c2_t3p = compute_chi2(x, star, datum)
            total_chi2 += c2_v2 + c2_t3a + c2_t3p

        # Add regularization
        reg = 0.0
        if regularizers:
            for name, weight, args in regularizers:
                if name == 'tv2':
                    r, _ = total_variation_l2(x, args)
                    reg += weight * r
                # ... etc

        return float(total_chi2 + reg)

    def gradient(x_np):
        x = jnp.array(x_np, dtype=jnp.float32)
        # Use JAX autodiff
        grad_fn = jax.grad(lambda x: objective_jax(x)) # need JAX-only version
        return np.array(grad_fn(x))

    # Use scipy L-BFGS-B
    result = minimize(
        objective, x_start, jac=gradient, method='L-BFGS-B',
        bounds=[(lower, upper)] * len(x_start),
        options={'maxiter': maxiter, 'disp': True},
    )

    return jnp.array(result.x)

```

**Better approach:** Define a single JAX function that returns (f, grad) and use `jax.value_and_grad`:

```

@jax.jit
def chi2_all_epochs(x, stars_data):
    """Compute total chi2 across all epochs. JIT-friendly version."""
    total = 0.0

```

```

for polyflux, polyft, visible_idx, v2, v2_err, t3phi, t3phi_err, ... in stars_data:
    # Inline the forward model
    ...
return total

```

```
chi2_and_grad = jax.value_and_grad(chi2_all_epochs)
```

The tricky part is making the StellarGeometry and OIData pytree-compatible for JAX. Consider using a flat tuple or namedtuple instead of dataclass for the inner loop data.

---

## 14. Coordinate Conventions and Gotchas

### Sign conventions in polyft

**THIS IS THE SINGLE MOST LIKELY SOURCE OF BUGS.**

The Julia code has multiple versions of `setup_polyft_single`, some commented out with **DIFFERENT** sign conventions on `ky`. The **production version** (lines 118-148, uncommented) uses:

```

kx = uv[1,:] * Float32(-pi / (180*3600000))
ky = uv[2,:] * Float32(-pi / (180*3600000)) # NEGATIVE for both

```

But **SOME** commented versions (e.g., line 102-103) use: `julia kx = uv[1,:] * Float32(-pi / (180*3600000)) ky = uv[2,:] * Float32( pi / (180*3600000)) # POSITIVE for ky!`

**USE THE PRODUCTION VERSION:** negative for both `kx` and `ky`. The sign of `ky` affects whether the image is flipped East-West. If your reconstructions look mirrored, this is probably why.

### Indexing: Julia 1-based → Python 0-based

Every index array from OIFITS or from neighbor computations needs to be shifted: - `indx_v2`, `indx_t3_1/2/3`: subtract 1 from Julia values - HEALPix pixel indices from `healpy`: already 0-based (good) - HEALPix pixel indices from ported Julia code: subtract 1

### HEALPix ordering

ROTIR uses **NESTED** ordering throughout. `healpy` defaults to **RING** ordering. Always pass `nest=True` to `healpy` functions.

### Float32 precision

The Julia code uses `Float32` throughout for performance. JAX defaults to `float32` on GPU. Be consistent — don't accidentally use `float64` anywhere in the hot path (it will be 2x slower on GPU and may cause type promotion issues).

### The `unit_xyz` after spherical recalculation

In `tessellation_healpix.jl` lines 20-23, the code recomputes `xyz` from the spherical coordinates **AFTER** getting the initial vertices from `pix2vec_nest`. This ensures the `xyz` and spherical representations are exactly consistent:

```

unit_xyz[:, :, 1] = sin.(unit_spherical[:, :, 2]).*cos.(unit_spherical[:, :, 3]);
unit_xyz[:, :, 2] = sin.(unit_spherical[:, :, 2]).*sin.(unit_spherical[:, :, 3]);
unit_xyz[:, :, 3] = cos.(unit_spherical[:, :, 2]);

```

Do the same in Python: after computing vertex spherical coords, recompute `xyz` from them.

### Asynchronous rotation ratio

The Julia `RocheParams` uses `async_ratio = P / rotation_period` (orbital period / stellar rotation period). Be careful: if the star is synchronously rotating, `rotation_period = P` and `async_ratio = 1.0`. The function `compute_potential_primary` receives `async_ratio` as  $\omega_{\text{rot}}/\omega_{\text{orb}}$ , which equals  $P_{\text{orb}}/P_{\text{rot}}$ .

Actually wait — looking more carefully at the Julia code:

```

In geometry_rochelobe.jl line 10: julia async_ratio = roche_parameters.P / roche_parameters.rotation_period

```

```

But in temperature_map_vonZeipel_roche_single line 271: julia async_ratio = parameters.rotation_period / parameters.P

```

**These are OPPOSITE conventions in the SAME codebase!** The Roche potential function (`compute_potential_primary`) uses  $\text{async\_ratio} = \omega_{\text{rot}}/\omega_{\text{orb}} = P_{\text{orb}}/P_{\text{rot}}$ . The gravity computation in `temperature_map_vonZeipel_roche_single` uses  $\text{rotation\_period}/P = P_{\text{rot}}/P_{\text{orb}}$ .

Looking at the potential formula (line 134): `julia  $\Omega_1 = 1/r + q/\text{dist} - q*r*\lambda*D + \text{async\_ratio}^2*(1+q)*r^2*(1-v^2)/2$`

The centrifugal term  $\text{async\_ratio}^2*(1+q)*r^2*(1-v^2)/2$  should increase with faster rotation. If  $\text{async\_ratio} = P_{\text{orb}}/P_{\text{rot}}$ , then faster rotation  $\rightarrow$  smaller  $P_{\text{rot}} \rightarrow$  larger  $\text{async\_ratio} \rightarrow$  larger centrifugal term. This is correct.

BUT in `compute_gravity_primary` (line 230): `julia  $g_x = -x.*\mu + q*(D.-x).*p + \text{async\_ratio}^2*(1+q)*x.-q*D$`

Here  $\text{async\_ratio}$  from `temperature_map_vonZeipel_roche_single` is  $P_{\text{rot}}/P_{\text{orb}}$ , which is the INVERSE.

**RESOLUTION:** In `temperature_map_vonZeipel_roche_single`, the variable is named `async_ratio` but computed as  $\text{rotation\_period}/P$ . Look at line 271 carefully—it's `parameters.rotation_period/parameters.P`. For synchronous rotation this is 1.0 either way. For async rotation, there's potentially a bug in the Julia code, or the gravity function uses a different convention.

**RECOMMENDATION:** In your Python code, be explicit. Define: `python  $F_{\text{rot}} = P_{\text{orb}} / P_{\text{rot}} \# = \omega_{\text{rot}} / \omega_{\text{orb}}$` .  $F_{\text{rot}} > 1$  means super-synchronous. Use  $F_{\text{rot}}$  consistently in both potential and gravity functions. Add a comment explaining the convention.

---

## 15. Testing Strategy

### Test 1: HEALPix vertices

```
def test_healpix_vertices():
    """Verify vertex computation against healpy.boundaries()."""
    for n in [2, 3, 4]:
        tess = tessellation_healpix(n)
        nside = 2**n
        for i in range(min(100, tess.npix)):
            # healpy boundaries returns 3xN array of xyz vectors
            boundaries = hp.boundaries(nside, i, step=1, nest=True)
            # Compare to our vertices (may need to match ordering)
            # Allow some tolerance for floating point
```

### Test 2: Polyflux consistency

```
def test_polyflux_sums_to_hemisphere():
    """For a uniform sphere, total polyflux  $\approx \pi r^2$  (projected disk area)."""
    tess = tessellation_healpix(4)
    params = SphereParams(inclination=0, position_angle=0, rotation_period=1, radius=1.0)
    star = create_star(tess, params, epoch=0.0)
    polyflux = compute_polyflux(star.projx, star.projy)
    total_area = jnp.sum(polyflux)
    expected = jnp.pi * 1.0**2 #  $\pi r^2$  for  $r=1$  mas
    assert abs(total_area - expected) / expected < 0.05 # 5% tolerance
```

### Test 3: Round-trip synthetic data

```
def test_roundtrip():
    """Generate synthetic data from known brightness, verify forward model recovers it."""
    # 1. Create uniform brightness sphere
    # 2. Generate UV coverage (random baselines)
    # 3. Compute model  $V^2$  and  $T3\phi$  (these are the "data")
    # 4. Run forward model with same brightness  $\rightarrow$  should match exactly
    # 5. Add noise, verify  $\chi^2 \approx$  number of data points
```

### Test 4: Gradient check

```
def test_gradient():
    """Verify JAX autodiff gradient matches finite differences."""
    # Use jax.test_util.check_grads or manual finite differences
```

## Test 5: Roche degenerates to sphere

```
def test_roche_degenerates():  
    """For  $q \rightarrow 0$  (no companion) and circular orbit, Roche  $\rightarrow$  sphere."""  
    # Set  $q = 0.001$ ,  $e = 0$ , verify radii are nearly constant
```

---

## 16. Build Order

Build and test in this order. Do NOT proceed to the next step until the current step passes tests.

### Step 1: datatypes.py

Write all dataclasses. No tests needed — these are just containers.

### Step 2: tessellation/healpix.py

Port the vertex computation. Test against `healpy.boundaries()`.

### Step 3: geometry/base.py — rotation only

Implement `rotation_matrix` and the rotation part of `create_star` for spheres only. Test: rotate a sphere, verify normals and visibility make sense.

### Step 4: forward\_model/polyft.py

Implement `compute_polyflux` and `compute_polyft`. Test: polyflux sums to  $\sim n \cdot r^2$  for a sphere.

### Step 5: forward\_model/observables.py

Implement `brightness_to_cvis`, `cvis_to_observables`, `compute_chi2`. Test: uniform brightness  $\rightarrow V^2=1$  everywhere,  $T3\phi=0$  everywhere (for a centered point source this is exact; for a resolved sphere it's approximate).

### Step 6: io/oifits\_reader.py

Implement OIFITS reader. Test: load a real file, verify array shapes and index ranges.

### Step 7: Integration test

Wire everything together: load OIFITS  $\rightarrow$  create geometry  $\rightarrow$  setup polyft  $\rightarrow$  compute chi2. Verify on real data that chi2 values are reasonable.

### Step 8: geometry/rapid\_rotator.py

Add rapid rotator radii and von Zeipel. Test: oblate shape, gravity darkening produces expected pattern.

### Step 9: geometry/orbits.py + geometry/roche.py

Add orbital mechanics and Roche potential. Test: Roche degenerates to sphere for  $q \rightarrow 0$ .

### Step 10: regularization/regularizers.py

Add TV, MEM, etc. Test: gradient check.

### Step 11: reconstruction/optimizer.py

Add L-BFGS-B wrapper. Test: recover synthetic spot on a sphere.

### Step 12: Multi-epoch support

Extend optimizer to sum chi2 across epochs. Test: synthetic multi-epoch recovery.

---

## Appendix: Julia-to-Python Quick Reference

| Julia                       | Python/JAX                                            |
|-----------------------------|-------------------------------------------------------|
| Float32                     | jnp.float32                                           |
| Complex{Float32}            | jnp.complex64                                         |
| Array{Float32}(undef, m, n) | jnp.zeros((m, n), dtype=jnp.float32)                  |
| abs2.(x)                    | jnp.abs(x)**2                                         |
| cis.(x)                     | jnp.exp(1j * x)                                       |
| sinc.(x)                    | jnp.sinc(x) (same convention: $\sin(\pi x)/(\pi x)$ ) |
| angle.(x)                   | jnp.angle(x)                                          |
| dot(a, b)                   | jnp.dot(a, b)                                         |
| a .* b (broadcast)          | a * b                                                 |
| A * b (matrix-vector)       | A @ b                                                 |
| A[i, :] (1-based)           | A[i-1, :] (0-based)                                   |
| findall(mask)               | jnp.where(mask)[0]                                    |
| mod2pi.(x)                  | x % (2 * jnp.pi)                                      |
| Threads.@threads for        | Not needed — JAX parallelizes on GPU                  |

---

## Final Notes for Claude Code / Sonnet

1. **Read this entire spec before writing any code.** Understand the data flow before implementing.
2. **Follow the build order.** Each step builds on the previous one. Don't skip ahead.
3. **Test each module independently.** Write a simple test script for each step. Don't accumulate untested code.
4. **When in doubt about math, reference the Julia source.** The function names and line numbers are provided.
5. **Don't over-engineer.** This is research code. Clean and correct beats elegant and complex.
6. **JAX gotchas:**
  - `jax.jit` requires static shapes — don't JIT functions where array sizes change
  - JAX arrays are immutable — use `at[].set()` for in-place updates, or just create new arrays
  - `jax.grad` only works on scalar-output functions
  - Be careful with integer operations inside JIT — use `jnp.int32` explicitly
7. **The `async_ratio` convention is a known hazard.** Pick one convention, document it, and be consistent. I recommend `F_rot = P_orb / P_rot` everywhere.
8. **The `polyft` sign convention is the other major hazard.** Both `kx` and `ky` should use the SAME negative scale factor. If images look mirrored, check this first.